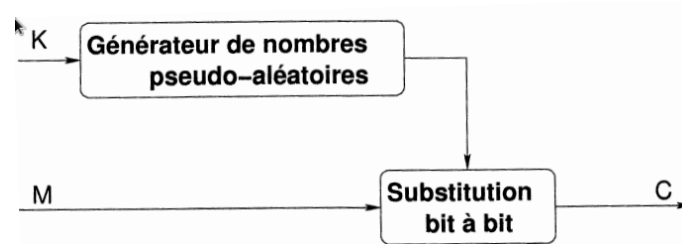


Chiffrement par flot

Registre à décalage à rétroaction linéaire

18 septembre 2026 – B. COLOMBEL

Les chiffrements symétriques par flot effectuent un traitement à la volée pour se rapprocher du modèle de VERNAM¹. Leur utilisation repose sur un générateur de nombres pseudo-aléatoires et un mécanisme de substitution bit-à-bit rapide comme l'opération « ou exclusif » ($\text{XOR } \oplus$) utilisée dans VERNAM.



On peut tout de suite remarquer en conséquence du principe de KERCKHOFFS que la sécurité d'un algorithme de chiffrement symétrique par flot repose sur la qualité du générateur de clé.

Parmi les exemples les plus connus (pas forcément les plus sûrs) de chiffrement par flot, on citera

- les LFSR (*Linear Feedback Shift Registers*);
- RC4 notamment utilisé dans le protocole SSL pour protéger les communications internet ou encore dans le protocole WEP utilisé dans les connexions WIFI;
- A5/3 utilisé dans les communications mobiles GSM
- etc.

Une propriété générale des chiffrements par flot est de générer un état interne à partir duquel des bits pseudo-aléatoires sont produits en un flot. L'expéditeur et le destinataire doivent synchroniser leurs états internes afin de pouvoir produire la même séquence de bits pseudo-aléatoires.

1 Quelques fonctions utiles

Commençons par charger les bibliothèques nécessaires au bon déroulement du TP.

```
import numpy as np
# import datetime as dt
```

Nous aurons besoin un « ou exclusif ». Cette opération en *python3* utilise l'opérateur `^`.

```
[2]: a = 60          # 60 = 0011 1100
      b = 13        # 13 = 0000 1101
      c = a ^ b ; c  # 49 = 0011 0001
```

```
[2]: 49
```

Cependant, nous n'utiliserons le OU EXCLUSIF que « bit à bit ». Nous pouvons remarquer que les résultats sont cohérents :

```
[3]: 0 ^ 0, 0 ^ 1, 1 ^ 0, 1 ^ 1
```

[3]:

1. Chiffre de Vernam ou masque jetable. Voir la page Wikipédia correspondante.

```
(0, 1, 1, 0)
```

Les messages sont d'abord convertis en suites de 0 et de 1 avant d'être transmis. Nous travaillerons donc maintenant non plus sur l'ensemble $\mathbb{Z}/26\mathbb{Z}$ mais sur $\mathbb{Z}/2\mathbb{Z}$ qui est un ensemble composé de deux valeurs 0 et 1.

Voici deux fonctions qui vous seront (certainement) utiles dans la suite du TP :

- `stringToBinary` convertit une chaîne de caractère en une suite binaire ;
- `binaryToString` permet de changer une suite binaire en chaîne de caractère.

```
def stringToBinary(msg):
    msg_bin = ""
    for i in bytearray(msg, encoding='ascii') :
        msg_bin = msg_bin + format(i, '08b')
    return msg_bin

def binaryToString(binary):
    msg = ""
    for i in range(0, len(binary), 8):
        byte_int = int(binary[i:i+8], 2)
        byte_char = chr(byte_int)
        msg = msg + byte_char

    return msg

print("En binaire :", stringToBinary("message en clair"))
print("En ascii :", binaryToString(stringToBinary("message en clair")))
```

2 Chiffrement de Vernam – masque jetable

Le masque jetable, aussi appelé « chiffrement de Vernam », repose sur le principe du ou exclusif bit à bit entre le message binaire à chiffrer et la clef de chiffrement (de même longueur).

Ainsi, étant donné une clef k de longueur n (donc $k \in (\mathbb{Z}/2\mathbb{Z})^n$)

$$E_k: (\mathbb{Z}/2\mathbb{Z})^n \rightarrow (\mathbb{Z}/2\mathbb{Z})^n$$
$$m \mapsto c = m \oplus k$$

Par exemple, avec $m = 11001100$ et $k = 10101011$, on aura $c = 01100111$ (Vérifiez par vous même !)

Le masque jetable garantit la sécurité des messages à condition qu'une clef ne serve qu'au chiffrement d'un seul message (d'où le "jetable" du nom) sinon, la cryptanalyse devient possible.

Exercice 1. Écrire une fonction `chiffrementVernam(msgBinaire, clef)` qui étant donné un message en clair binaire `msgBinaire`, et une suite binaire `clef` de même longueur que le message retourne le message chiffré correspondant.

Testez votre fonction avec

```
try:
    assert chiffrementVernam(stringToBinary("vernam"), \
        "11001100110011001100110011001100110011001100110011001100") == \
        "101110101010100110111101010001010110110100001"
    print("chiffrementVernam : OK")
except:
    print("chiffrementVernam : ERREUR")
```

Le déchiffrement s'opère en exécutant la même opération : $m = c \oplus k$

Exercice 2. Quel clair (en ascii) représente le chiffré `101000101010110110101001101111010111000` codé avec la clef `110011001100110011001100110011001100110011001100` ?

Écrire le bout de code permettant de le déchiffrer.

3 Registre à décalage à rétroaction linéaire

En pratique, il existe deux inconvénients majeurs au principe du chiffrement par masque jetable :

1. Les clefs doivent faire la même longueur que le message à chiffrer. Transmettre des clefs aussi longues est un problème en soi.
2. Pour assurer la sécurité des messages, il faut que la clef soit choisie aléatoirement. Or, nous ne savons pas comment produire du vrai aléatoire avec un ordinateur.

Les *registres à décalage à rétroaction linéaire* ou *LFSR* (pour linear feedback shift register) permettent de palier partiellement ces deux inconvénients en générant une suite binaire proche de l'aléatoire véritable à partir de quelques bits appelés *graine* (seed en anglais). Il suffit donc de transmettre la graine ainsi que la fonction interne du LFSR, et non plus la suite entière, au receveur du message pour qu'il puisse déchiffrer le message.

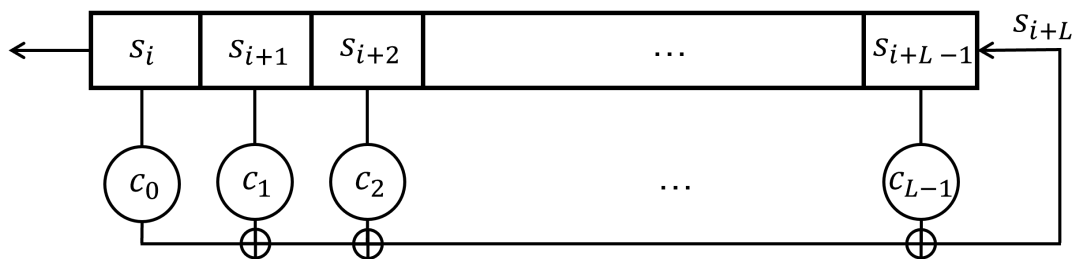
Un LFSR binaire de longueur L est composé :

- d'un registre à décalage contenant une suite de L bits $(s_i, s_{i+1}, \dots, s_{i+L-1})$ décrivant l'état interne du registre ;
- d'une fonction de rétroaction linéaire permettant de calculer la valeur du bit suivant à insérer dans le registre.

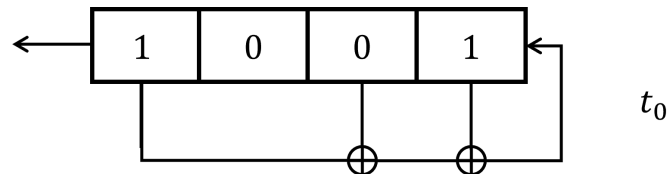
À chaque top d'horloge, le bit s_i constitue la sortie du registre, et les autres sont décalés ; le nouveau bit s_{i+L} , placé dans la cellule rendue libre du registre, est donné par une fonction linéaire : $s_{i+L} = c_0 s_i \oplus c_1 s_{i+1} \oplus \dots \oplus c_{L-1} s_{i+L-1}$ où les coefficients c_i sont binaires.

On appelle *suite chiffrante* la concaténation des sorties du registre. C'est cette suite qui servira ensuite de clef de chiffrement.

On peut représenter ce LFSR de la manière suivante :



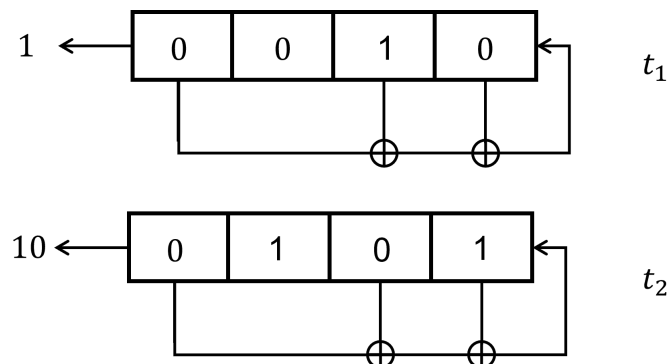
Ci-dessous un exemple de LFSR de longueur $L = 4$ initialisé avec la graine 1001 à t_0 :



Les coefficients de la fonction de rétroaction sont : $c_0 = 1, c_1 = 0, c_2 = 1, c_3 = 1$.

Le bit inséré à droite est donc calculé grâce à la formule : $s_{i+4} = s_i \oplus s_{i+2} \oplus s_{i+3}$.

Nous avons déroulé les 2 premières itérations (t_1 et t_2) du registre. A l'étape t_1 , la sortie est le bit situé le plus à gauche du registre (de valeur 1). A l'étape t_2 , c'est le bit 0 qui sort permettant de créer la suite chiffrante 10 en considérant la sortie de l'étape précédente.



Exercice 3. Écrire une fonction `etatSuivant(etat,coeff)` qui prend une liste binaire correspondant à l'état interne du registre ainsi qu'une liste des coefficients de rétroaction non nuls (i.e. indices des cases sur lesquelles faire le `xor`) et renvoie l'état suivant du registre et le bit de sortie.

Testez votre fonction avec :

```
try:
    assert etatSuivant([1,0,0,1],[0,2,3]) == ([0, 0, 1, 0],1) #Exemple précédent t1
    assert etatSuivant(etatSuivant([1,0,0,1],[0,2,3])[0],[0,2,3]) == ([0, 1, 0, 1],0) #Exemple précédent t1
    print("etatSuivant : OK")
except:
    print("etatSuivant : ERREUR")
```

Exercice 4. Quelle sera la suite chiffrante à t14? Que remarquez vous?

Exercice 5. Écrire une fonction `suite_LFSR(graine,coeff, n)` qui prend en entrée une liste binaire correspondant à la graine du registre, une liste des coefficients de rétroaction non nuls et la longueur souhaitée de la suite chiffrante et qui renvoie la suite chiffrante binaire sous forme d'une chaîne de caractère.

Testez votre fonction avec

```
try:
    assert suite_LFSR([1,0,0,1],[0,2,3],14) == "10010111001011"
    print("suite_LFSR : OK")
except:
    print("suite_LFSR : ERREUR")
```

Exercice 6. Ecrire une fonction `chiffrementLFSR(msgAscii, graine, coeff)` qui déroule l'ensemble du processus de chiffrement par masque jetable généré par LFSR et retourne la version binaire du message chiffré.

Testez votre fonction avec

```
try:
    assert chiffrementLFSR("naert",[1,0,0,1],[0,2,3]) == "1111100101001111001110011100101100000110"
    print("chiffrementLFSR : OK")
except:
    print("chiffrementLFSR : ERREUR")
```

Exercice 7. Écrire une fonction `dechiffrementLFSR(msgChiffBinary, graine, coeff)` qui déroule l'ensemble du processus de déchiffrement et retourne la version ascii du message en clair.

Testez votre fonction avec

```
try:
    assert dechiffrementLFSR("1111100101001111001110011100101100000110",[1,0,0,1],[0,2,3]) == "naert"
    print("dechiffrementLFSR : OK")
except:
    print("dechiffrementLFSR : ERREUR")
```

4 Générateur à signal d'arrêt

En pratique, la suite chiffrante produite par un unique LFSR n'est pas assez complexe pour servir de clef de chiffrement. Nous avons vu notamment à la question 4 que les LFSR pouvaient produire des suites chiffrantes périodiques donc loin d'une suite réellement aléatoire. En réalité, toute suite chiffrante produite par un unique LFSR est ultimement périodique.

Définition 1

Soit une suite $s = (s_0, s_1, s_2, \dots)$ avec pour tout $i \in \mathbb{N}$, $s_i \in \mathbb{Z}/2\mathbb{Z}$

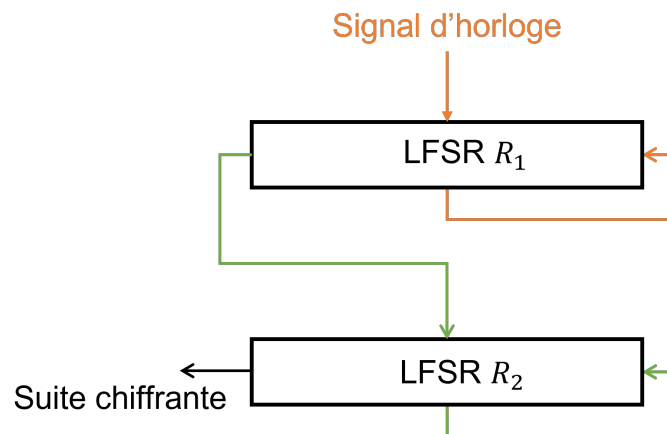
1. On dit que s est *périodique* de période T si $s_i = s_{i+T}$ pour tout $i \geq 0$

2. On dit qu'une suite s est *ultimement périodique* de période T si $s_i = s_{i+T}$ pour tout i supérieur ou égal à un certain rang appelé i_0 .

Ainsi, une suite périodique est ultimement périodique ($i_0 = 0$) mais la réciproque est fausse.

Pour générer des suites chiffrantes utilisables dans un système cryptographique, les LFSR sont utilisés comme briques de base de système plus complexes : plusieurs LFSR peuvent être combinés pour atteindre la complexité souhaitée. Dans ce TP, nous étudierons un exemple d'un tel générateur : le **générateur à signal d'arrêt**.

Le générateur à signal d'arrêt (GSA) est un exemple de registre à décalage irrégulier (1984). Il utilise la sortie d'un premier LFSR R_1 pour contrôler l'horloge d'un second LFSR R_2 . Plus précisément, R_1 est un LSFR "normal" dont le décalage est commandé par un signal d'horloge (en orange sur l'image ci-dessous) mais R_2 ne change d'état à l'instant t que si la sortie de R_1 (en vert) est égale à 1 à l'instant $t - 1$, autrement dit si la sortie de R_1 est égale à 0 à l'instant $t - 1$, alors R_2 n'est pas décalé et le bit de sortie à l'instant t est donc toujours égal au bit de sortie à l'instant $t - 1$.



Exercice 8. Écrire une fonction `suite_gsa(graineR1,coeffR1, graineR2, coeffR2, n)` qui renvoie la suite chiffrante binaire de longueur n générée par un générateur à signal d'arrêt composé de R_1 et R_2 .

Testez votre fonction avec :

```
print("10 premiers termes de la suite chiffrante : ", \
      suite_gsa([1,0,1,0,1,1,0,0],[0, 3, 5],[1,0,1,0,1,0,1,0], [0, 2, 5, 6],10))

try:
    assert suite_gsa([1,0,1,0,1,1,0,0],[0, 3, 5],[1,0,1,0,1,0,1,0], [0, 2, 5, 6],10) == \
        "1001101111"
    print("suite_gsa : OK")
except:
    print("suite_gsa : ERREUR")
```